Deliverable 2

Group 7

Github:
https://github.com/SarahAsghar/EECS3311-YorkU-Lab-Equipment-Reservation-Platform-Group19

Task 3 Video: https://youtu.be/U827O9FEWn8

Task 1: Parts a-c completed by Seyed Parsa Shoushtari

Part a)

Justification for Choosing Six Design Patterns

Six design patterns were selected for the redesigned YorkU Lab Equipment Reservation Platform: MVC, Singleton, Factory Method, Strategy, Observer, and State. These patterns were chosen because they directly address structural and behavioral challenges in the system requirements and improve the overall organization of the application.

The **MVC (Model View Controller)** pattern was chosen as the main architectural structure for the Java GUI application. The system includes several different concerns such as persistent data (users, equipment, reservations, and payments), graphical user interface screens (login, registration, reservation management, and administration), and application logic that enforces reservation rules and access control. Without a clear separation of responsibilities, changes to the user interface could easily affect the system logic or data model. MVC organizes the system into three layers. The **Model**contains the domain classes such as User, Equipment, Reservation, and Payment. The **View** represents the graphical interface screens. The **Controller** processes user input and coordinates operations between the interface and the model. This structure keeps the system modular and easier to maintain.

The **Singleton** pattern was chosen for the **DatabaseManager** class. The system uses CSV files to simulate database storage, which means that all data access must be coordinated carefully. If multiple database manager objects accessed the same files simultaneously, it could lead to inconsistent data or file corruption. The Singleton pattern ensures that there is only one DatabaseManager instance in the entire application. All controllers access persistent

data through this shared instance using the getInstance() method. This centralizes file input and output operations and keeps data access consistent.

The **Factory Method** pattern was used to handle the creation of certain domain objects. The system supports multiple user categories and multiple payment methods. For example, users may be students, faculty members, researchers, or guests, and payments may be made through credit cards, debit cards, institutional accounts, or research grants. Instead of creating these objects directly in the controllers, the system uses **UserFactory** and **PaymentFactory** classes to generate them based on the selected type. This keeps object creation logic centralized and makes it easier to add new user types or payment methods later without modifying existing controller code.

The **Strategy** pattern was selected for payment processing. Requirement 10 states that the system must support several payment methods. Each payment method may involve different validation steps or processing behavior. Placing all of this logic inside the Payment class would create complex conditional code and make the class difficult to maintain. The Strategy pattern solves this by defining a common **PaymentStrategy** interface. Concrete strategies such as CreditCardPaymentStrategy, DebitCardPaymentStrategy, InstitutionalAccountPaymentStrategy, and ResearchGrantPaymentStrategy implement the specific behavior for each method. The Payment class simply delegates the processing operation to the selected strategy, making the system easier to extend.

The **Observer** pattern was chosen to support equipment sensor updates. In Deliverable 1, the Equipment Sensor System was modeled as an external component that reports equipment usage and status changes. Instead of having the system repeatedly check the sensor for updates, the Observer pattern allows the sensor to notify interested components when a change occurs. In the design, **EquipmentSensor** acts as the subject and sends updates to registered observers. The **EquipmentStatusObserver** receives these updates and forwards them to components such as Equipment and ReservationController. This event driven design allows the system to react automatically when equipment status changes.

The **State** pattern was selected to manage the lifecycle of a reservation. A reservation can move through several stages including pending, confirmed, active, completed, cancelled, and no show. The operations allowed on a reservation depend on its current state. For example, a reservation can only be extended while it is active, and a no show occurs if the user does not

arrive within the allowed time period. Instead of handling these rules with complex conditional statements inside the Reservation class, the system defines a **ReservationState** interface and separate classes for each state. The Reservation object maintains a reference to its current state and delegates state specific behavior to it. This keeps lifecycle logic organized and easier to maintain.

Together, these six design patterns address several key design challenges in the system. MVC improves separation between the interface, logic, and data. Singleton ensures consistent data access. Factory Method centralizes object creation. Strategy supports multiple payment behaviors. Observer enables event driven sensor updates. State manages reservation lifecycle behavior. Combined, these patterns produce a system design that is flexible, modular, and easier to extend as new requirements are added.

Part b)

Updated Class Diagram

The system was redesigned to incorporate six design patterns while preserving the core domain model developed in Deliverable 1. The updated class diagram integrates the original domain entities with additional supporting classes required to implement the selected design patterns. This redesign improves modularity, maintainability, and extensibility while maintaining the original system functionality.

The main domain classes are **User, Equipment, Reservation, Payment, and UsageRecord**. These classes represent the core data entities of the lab equipment reservation platform. Two specialized roles, **HeadLabCoordinator** and **LabManager**, extend the base **User** class. These roles represent the administrative users responsible for approving accounts and managing equipment within the system.

To support the six design patterns, several additional classes were introduced. These include controller and view classes for MVC separation, a **DatabaseManager** implemented using the Singleton pattern for centralized data access, factory classes for controlled object creation, strategy classes for handling different payment methods, observer components for sensor based equipment updates, and state classes to manage the reservation lifecycle.

The following diagrams present the full updated class structure, followed by focused sub diagrams that highlight how each design pattern is implemented in the system.

Figure 1 Overall Updated Class Diagram

The overall class diagram shows the full architecture of the redesigned system. It integrates the core domain classes with the additional components required by the design patterns.

Several important relationships are illustrated. A **User** may create multiple **Reservations**. Each **Reservation** is associated with one **Equipment** item and linked to exactly one **Payment**. Equipment may generate multiple **UsageRecords** through sensor based status updates. All controllers interact with the **DatabaseManager** Singleton to access persistent storage.

The diagram also shows how pattern related classes such as factories, strategies, observers, and state classes interact with the core domain model.

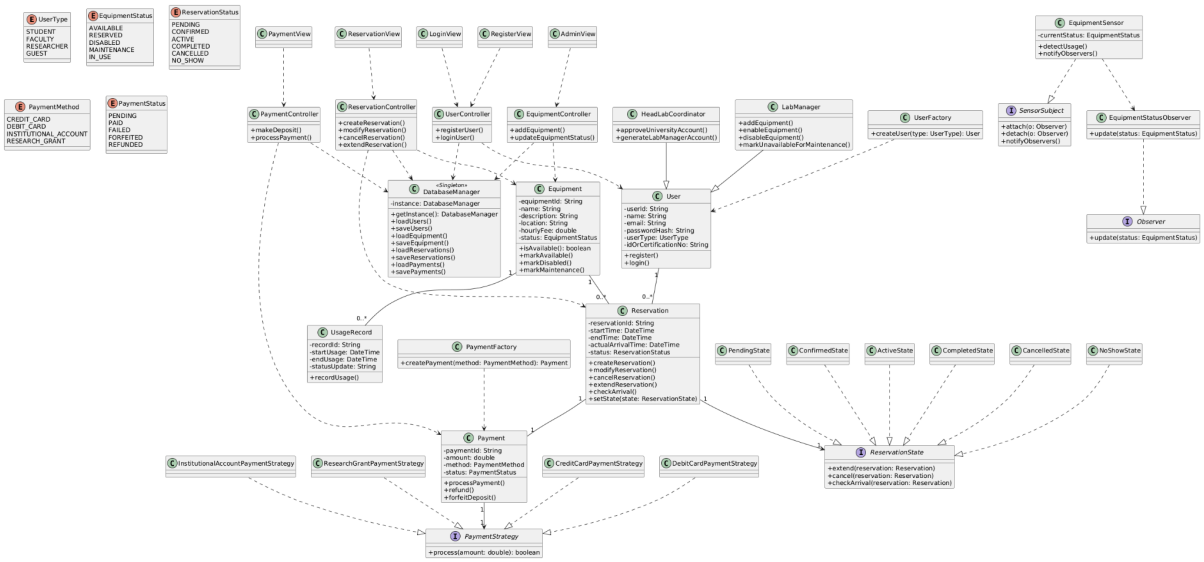


Figure 2 MVC Pattern

The MVC diagram illustrates how the system separates the graphical interface, application logic, and data model into three distinct layers.

The **View layer** contains the graphical interface components, including **LoginView, RegisterView, ReservationView, AdminView, and PaymentView**. These classes represent the GUI screens that users interact with.

The **Controller layer** processes user input and coordinates system behavior. This layer includes **UserController, ReservationController, EquipmentController, and PaymentController**.

The **Model layer** contains the domain classes **User, Reservation, Equipment, Payment, and UsageRecord**, which represent the system data and business rules.

Views communicate with controllers, and controllers interact with the model. This structure ensures that user interface changes do not affect the underlying business logic or data model.

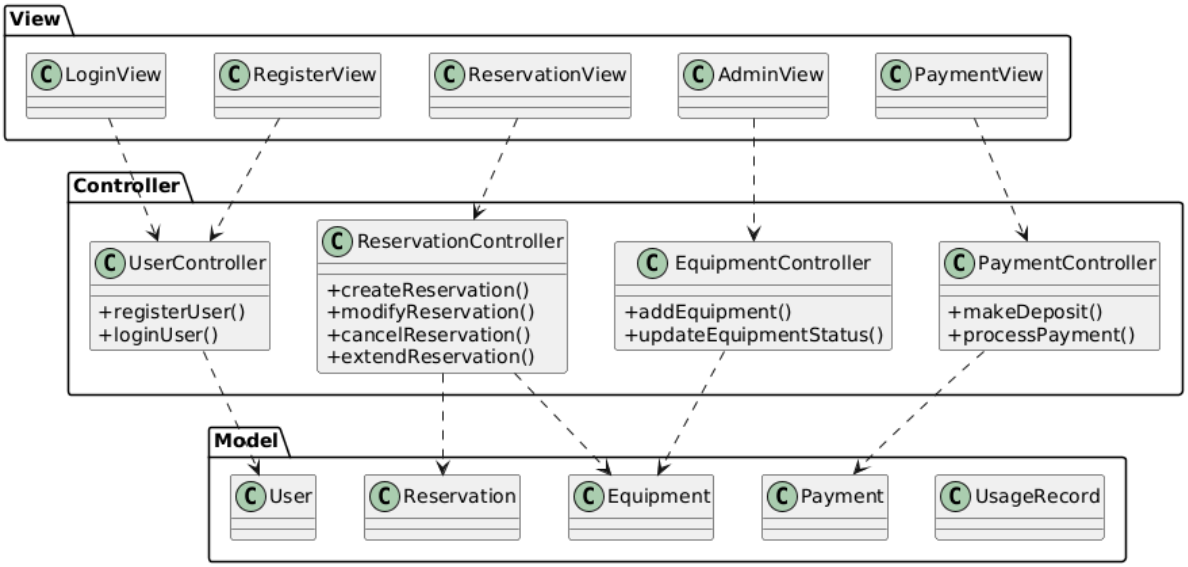


Figure 3 Singleton Pattern

The Singleton pattern is implemented using the **DatabaseManager** class. Since the system uses CSV files to simulate database storage, all components must access the same shared data source.

The **DatabaseManager** provides a single access point for loading and saving system data such as users, equipment, reservations, and payments. Controllers access the database manager using the **getInstance()** method, which guarantees that only one instance exists during the lifetime of the application.

This design keeps file access centralized and prevents inconsistent reads or writes from multiple controllers.

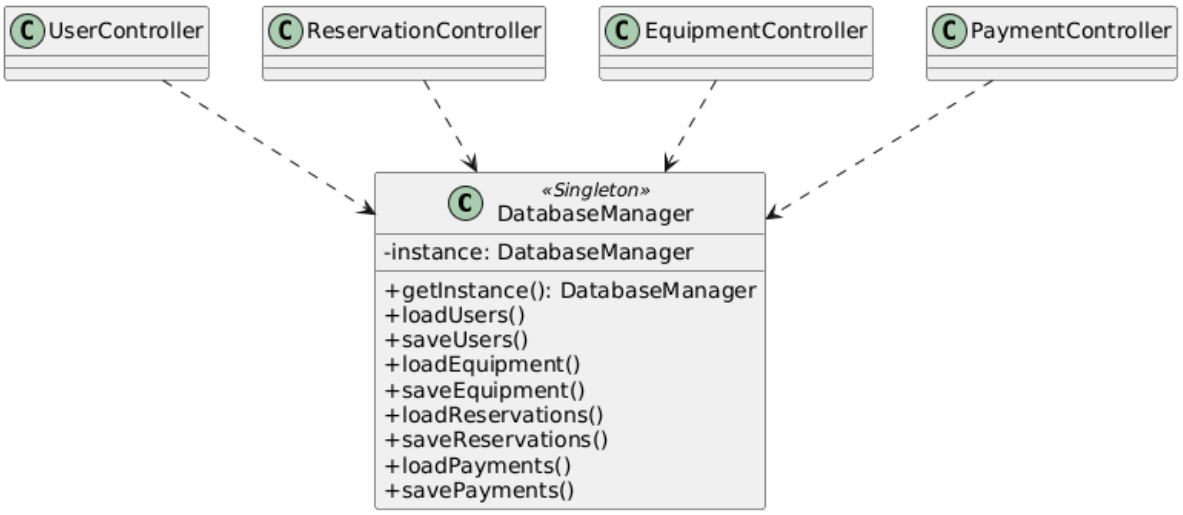


Figure 4 Factory Method Pattern

The Factory Method pattern is used to centralize the creation of certain objects within the system.

Two factory classes are included: **UserFactory** and **PaymentFactory**. The **UserFactory** creates **User** objects based on the selected **UserType**, such as student, faculty, researcher, or guest. The **PaymentFactory** creates **Payment** objects based on the selected **PaymentMethod**, such as credit card, debit card, institutional account, or research grant.

Using factory classes avoids spreading object creation logic across multiple controllers and makes the system easier to extend if additional user types or payment methods are added later.

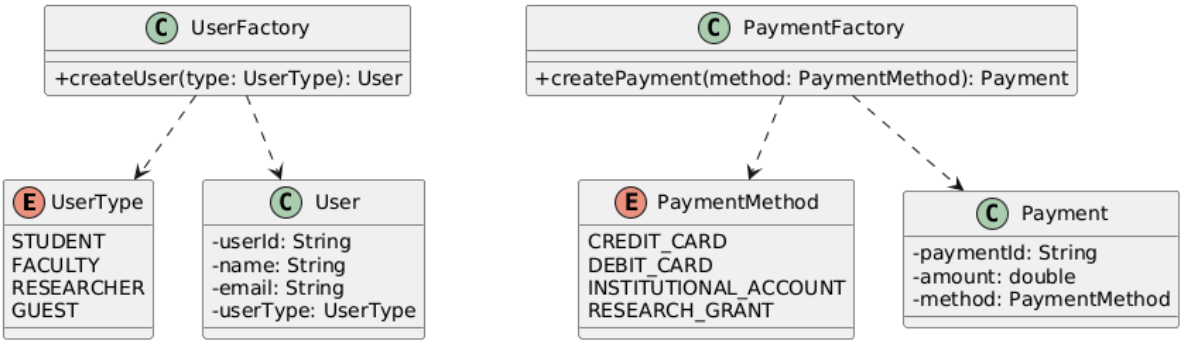


Figure 5 Strategy Pattern

The Strategy pattern is used to support multiple payment processing methods.

The **PaymentStrategy** interface defines a common method called **process(amount)**. Several concrete classes implement this interface, including **CreditCardPaymentStrategy, DebitCardPaymentStrategy, InstitutionalAccountPaymentStrategy, and ResearchGrantPaymentStrategy**.

The **Payment** class stores a reference to a **PaymentStrategy** object and delegates the payment processing logic to the selected strategy. This allows different payment methods to be handled independently without placing complex conditional logic inside the Payment class.

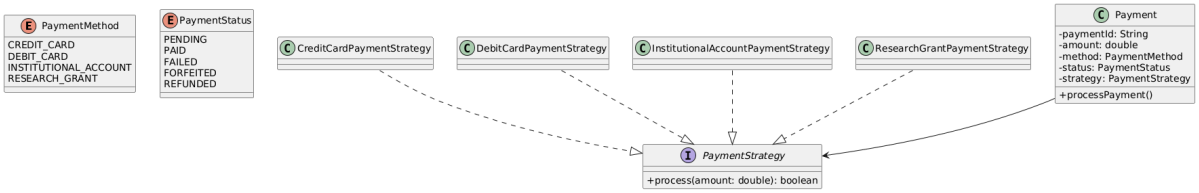


Figure 6 Observer Pattern

The Observer pattern is used to handle equipment status updates generated by sensors.

The **EquipmentSensor** acts as the subject and implements the **SensorSubject** interface. This interface provides the methods **attach()**, **detach()**, and **notifyObservers()**.

When the sensor detects equipment usage through **detectUsage()**, it notifies all registered observers by calling **update(status)**.

The **EquipmentStatusObserver** implements the **Observer** interface and forwards these updates to relevant system components such as the **Equipment** class and the **ReservationController**. This allows the system to respond automatically to sensor events, for example updating equipment status or detecting a no show situation.

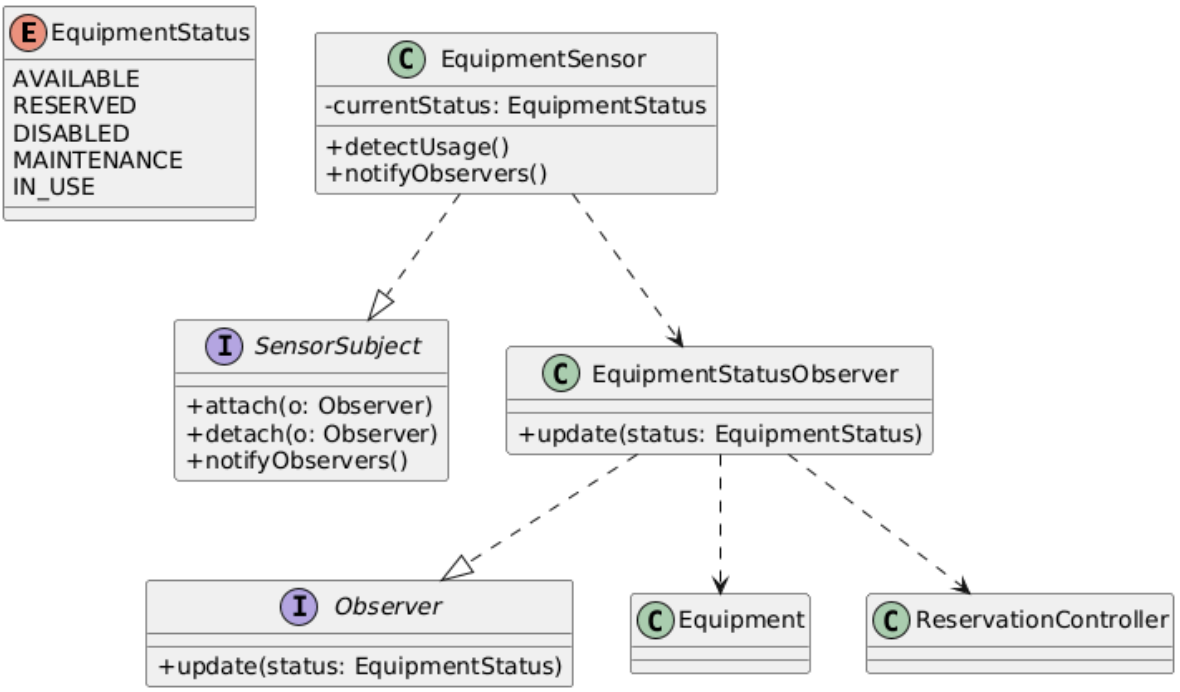


Figure 7 State Pattern

The State pattern is used to model the lifecycle of a reservation.

The **ReservationState** interface defines the operations **extend()**, **cancel()**, and **checkArrival()**. These operations are implemented differently depending on the current reservation state.

Concrete state classes include **PendingState, ConfirmedState, ActiveState, CompletedState, CancelledState, and NoShowState**.

The **Reservation** class maintains a reference to the current **ReservationState** and delegates state specific behavior to that object using **setState()**. This ensures that only valid operations can occur at each stage of the reservation lifecycle. For example, reservation extension is only allowed when the reservation is active, and deposit forfeiture occurs when the system transitions to the NoShowState.

This approach keeps lifecycle behavior organized and prevents large conditional statements inside the Reservation class.

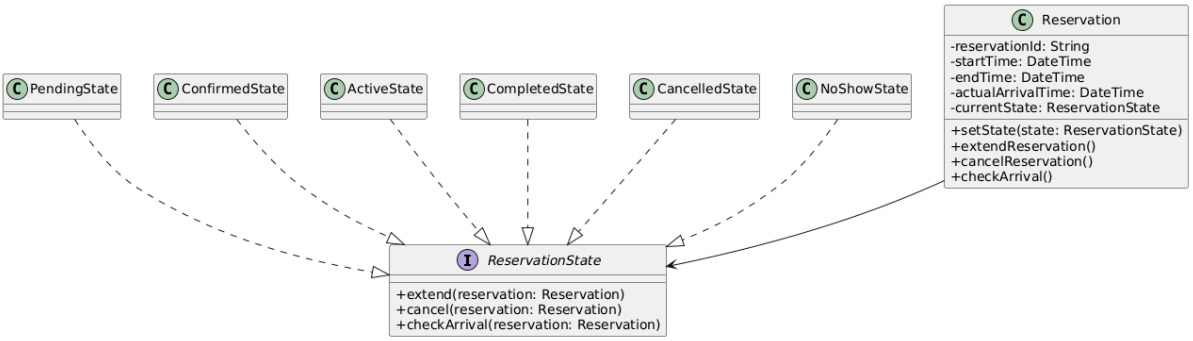
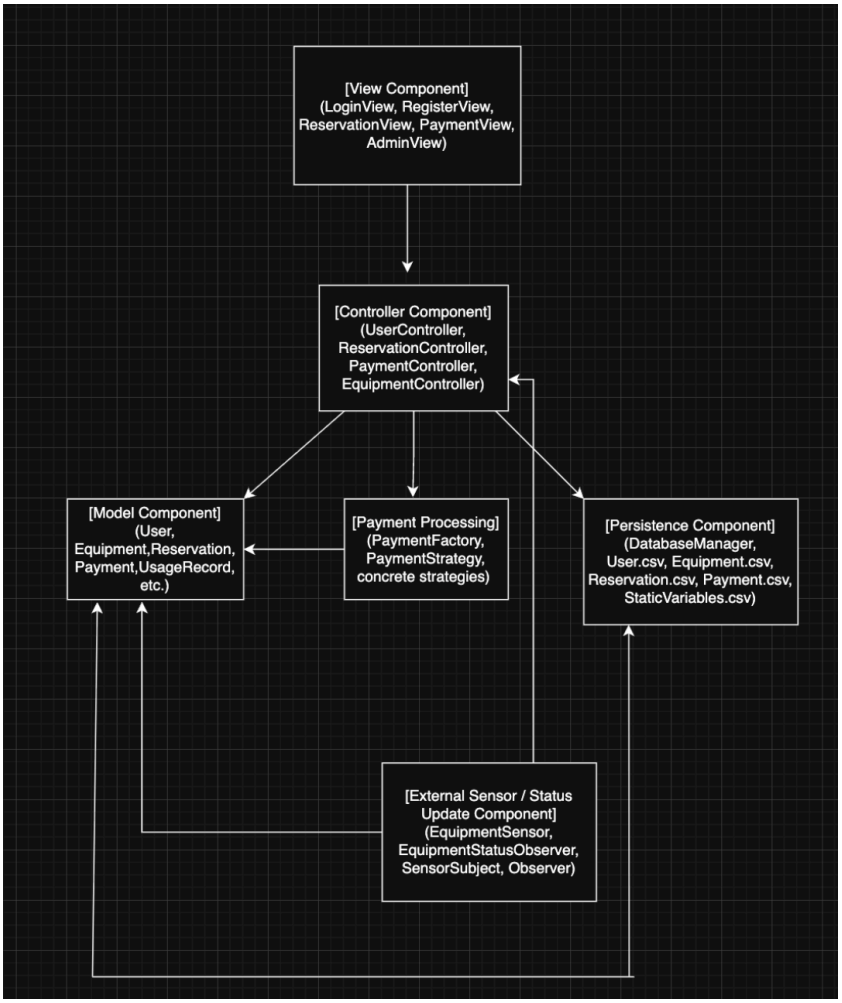


Figure 8

Task 1(d): Component Decomposition and Interactions



Task 1(e): Justification for Component Decomposition and Interactions

We divided our system into several main components to keep the design organized and manageable. The project includes different tasks such as user interaction, system logic, data storage, payment handling, and equipment status tracking. By separating these into components, we avoided grouping everything into one large, confusing structure. This approach made the system easier to understand, test, and modify later if we add new features.

The View component contains the user interface screens, including login, registration, reservation, payment, and admin pages. This part of the system is responsible for displaying information to the user and collecting their input. It does not manage business logic directly. Instead, the Controller component acts as the middle layer that receives user actions from the view and decides what happens next. For example, when a user makes a reservation or processes a payment, the controller validates the request and calls the appropriate model or persistence functions.

The Model component holds the core data of the system. It includes classes like User, Equipment, Reservation, Payment, and UsageRecord. These classes store the system's information and enforce the main rules of the application. We separated the Persistence component so that all file operations are handled in one place using the DatabaseManager and CSV files. This keeps data storage consistent and prevents controllers from managing file operations directly.

We also created a separate Payment Processing component because payment behavior can change based on the payment method. By grouping the factory and strategy classes into their own component, we made it easier to support different payment types without affecting the rest of the system. Additionally, we separated the External Sensor/Status Update component since equipment usage tracking follows an event-based design. Sensors detect changes in equipment status and notify the system when updates occur, helping keep the reservation system accurate and up to date.

Overall, the interactions between components follow a clear flow. The view sends requests to the controller, the controller updates the model and communicates with persistence when data needs to be saved or retrieved, payment logic is managed through the payment processing component, and equipment status updates are handled through the sensor component. This structure keeps responsibilities clearly separated and helps the system stay organized as it grows.

Task 4 : AI Assistant Adaptation Report

During the redesign and implementation of our Lab Equipment Reservation Platform, AI tools were used occasionally to support development, especially when we needed quick examples or help troubleshooting. However, most of the final design decisions and debugging were done manually by the team. We treated AI more like a helper for brainstorming or drafting code rather than something that replaced our own work. In several cases, the AI suggestions were incomplete or slightly incorrect, so we had to review and adjust them to fit our system requirements and architecture.

Different team members used different tools depending on the task. The main tools used were ChatGPT-4, ChatGPT-5, and DeepSeek. Each person used AI in small ways to assist with their specific part of the project.

Role of AI in Individual Tasks

Team Member	Task / Component	AI Tool Used	What We Asked the AI	AI Output Summary	How We Used It	Final Decision
Sarah	Payment Controller Implementation, Some UI development	DeepSeek	Generate a basic payment processing controller to handle payment confirmation and deposit logic, Generate appropriate components to be used in the UI	AI produced a working PaymentController class with payment validation and processing methods, and proper components and actions to be used in the UI	The generated code helped speed up the initial structure, but some validation checks were missing and variable names were inconsistent with our project design	Modified,structure kept, logic corrected manually
Parsa	Design Pattern Selection	ChatGPT-4	Suggest design patterns suitable for a reservation system with equipment management	AI suggested MVC, Factory, Strategy, Observer, Singleton, and State patterns	The suggestions matched our design needs and helped confirm our approach	Accepted with minor refinements
Sarah	User and Payment	ChatGPT-5	Generate example	AI produced	We adapted the logic to	Modified

	Object Creation		factory classes for creating user and payment objects dynamicall y	UserFactor y and PaymentFa ctory examples	match our system classes and enums	
Nabeela	Requireme nt Verification and Integration	ChatGPT-5	Check whether all system requirement s were covered in the implementa tion	AI provided a checklist comparing requirement s to system components	Used as a validation step to confirm coverage before submission	Accepted after manual verification
Team	Sensor System Logic (Req5)	ChatGPT-4	Explain how sensors can notify the system when equipment usage changes	AI suggested using an Observer pattern to send updates to controllers	This idea aligned with our existing architecture and was used to finalize the sensor update logic	Accepted

Section B : Arguments and Resolution (Disagreement with AI)

Situation	**AI Suggestion**	**Issue Identified**	**Reasoning Process**	**Final Resolution**
Payment Controller Validation	AI suggested automatically approving payments once the amount was entered	This ignored validation for invalid payment methods and insufficient balances	We reviewed system requirements and determined that payment verification must	Payment validation logic was rewritten manually to include method checks and error

			occur before approval	handling
Sensor Requirement (Req5)	AI initially suggested polling sensors repeatedly to check equipment status	This approach would cause unnecessary system load and did not match the event-driven design described in the architecture	We reviewed the Observer pattern design and determined that sensors should notify the system only when changes occur	Implemented event-based notification using the Observer pattern
Reservation State Transitions	AI suggested allowing reservation extension in any state	This violated system rules because only active reservations should be extended	We checked the reservation lifecycle logic and state definitions	Restricted extension functionality to the Active state only

Section C Verification of AI Responses

AI-generated suggestions were verified through manual testing, UML diagram review, and requirement mapping. Each output was checked against system requirements and design constraints to ensure correctness.

Verification Area	**Method Used**	**Example**	**Result**
UML Diagram Consistency	Compared AI suggestions with official UML standards and class relationships	AI generated a diagram where controllers directly accessed data files instead of the database manager	Diagram was corrected to route all data access through the Singleton DatabaseManager
Design Pattern Correctness	Reviewed pattern structure against lecture notes and system architecture	AI suggested placing payment logic inside the Payment class instead of using a Strategy pattern	Strategy pattern implementation was preserved
Requirement Coverage	Tested system features using sample scenarios	Verified that equipment status updates triggered reservation updates	Requirement confirmed as satisfied
Code Functionality	Executed program with test data	Payment processing initially failed for	Error corrected by updating validation

		institutional accounts	conditions

Section D Requirement Coverage (Req1–Req10)

The system was tested to ensure that each functional requirement was implemented correctly. AI tools were used to assist with debugging and validation, but final confirmation was performed manually.

Requirement	Description	Implementation Summary	AI Role	Verification Method
Req1	User registration and login	Implemented using MVC structure with UserController and LoginView	AI suggested validation logic	Tested user login scenarios
Req2	Equipment browsing	Equipment list displayed through ReservationView	AI provided example UI structure	Verified equipment display
Req3	Reservation creation	Users can reserve equipment with date and time selection	AI helped generate initial method structure	Tested reservation workflow
Req4	Payment processing	PaymentController processes deposits using Strategy pattern	AI generated initial controller template	Tested multiple payment methods
Req5	Sensors track equipment usage and send updates	EquipmentSensor detects usage and notifies observers when status changes	AI suggested Observer pattern logic	Simulated equipment usage events
Req6	Reservation modification	Users can extend or cancel reservations	AI provided example method signatures	Tested reservation updates
Req7	Reservation lifecycle management	State pattern manages reservation status transitions	AI suggested state structure	Verified state transitions
Req8	Administrative management	LabManager can manage equipment records	AI provided sample admin interface logic	Tested admin actions

Req9	Data persistence	CSV files store user and reservation data through DatabaseManager	AI suggested file handling structure	Verified data loading and saving
Req10	Multiple payment methods	Strategy pattern supports credit, debit, and institutional payments	AI generated initial strategy classes	Tested all payment options

Section E :Example of AI Error That Required Human Fix

One issue occurred during the implementation of the payment controller.

The AI-generated code initially processed payments without verifying whether the selected payment method matched the user's account type. This caused incorrect payment approvals during testing. After reviewing the system requirements, we manually added validation checks to ensure that each payment method was properly verified before confirmation. This correction improved system reliability and ensured that the payment process followed the expected business rules.

Section F: Summary of AI Usage

AI tools were used primarily to support development efficiency and clarify design concepts. The team maintained control over system architecture, debugging, and validation. All final implementations were reviewed and tested manually to ensure compliance with system requirements.

Overall, AI assistance reduced development time for repetitive coding tasks and helped identify potential design improvements, while human decision-making ensured correctness and system reliability.

Name	**Student Number**	**Part(s) Completed**
Seyed Parsa Shoushtari	219615574	Task 1 - a,b,c
Sarah Asghar	219428127	Task 2
Nabeela Ansari	219730662	Task 4, Task1: d,e
Sadia Jahan Shifa	219302744	Task 2, Task 3
Doluwamu Olubiremi	218801563	Task 2